

CSC209 Project Documentation: Chat Server System

Noa Higuchi

5.1 Project Overview

My project implements a multi-user chat server (Category 2). The system consists of a central server that handles multiple concurrent client connections using TCP sockets and the `select()` system call for non-blocking I/O.

Key Features:

- **Multi-user chat:** Multiple clients can connect at the same time and exchange messages
- **Channel-based communication:** Users can create and join different chat channels
- **User authentication:** Clients must join with a unique username
- **Color customization:** Users can change their display color
- **Message history:** Server maintains recent message history per channel
- **Graceful disconnection:** Clean shutdown handling for both clients and server

User Interaction:

Users interact with the system through the terminal. After starting the client program, users can:

1. Join the chat with a username using `/join <username>` (see `client_run()` in `src/client.c:512-525`)
2. Send text messages with the Enter key (handled by `process_user_input()` in `src/client.c`)
3. Manage channels with `/create`, `/channel`, and `/list` commands (implemented in `handle_channel_*` functions in `src/server.c`)
4. Change their username's color with `/color <color>` (see `handle_color_message()` in `src/server.c`)
5. Exit the program with `/quit` (calls `send_leave_message()` in `src/client.c`)

Input/Output:

- **Server:** Listens on TCP port 4242 by default (configurable), accepts connections, routes messages
- **Client:** Connects to server, provides command-line interface for users
- **Protocol:** Binary message format with headers and structured payloads

5.2 Build Instructions

Prerequisites:

- GCC compiler
- Standard C library
- Unix-like operating system

Build Commands:

make

Or build with a specific port:

PORT=9999 make

Running the Project:

Terminal 1: Start the server

`./server`

(defaults to 4242 but can change port with the `-p` flag)

Terminal 2: Start a client (repeat for additional clients)

`./client`

(optional flags: `-h <host-ip>` `-p <port>`)

Command Line Arguments:

Server: (parsed in `src/main_server.c:main()`)

- `-p PORT` - Port number to listen on (default: 4242)
- `-h` - Show help message

Client: (parsed in `src/main_client.c:main()`)

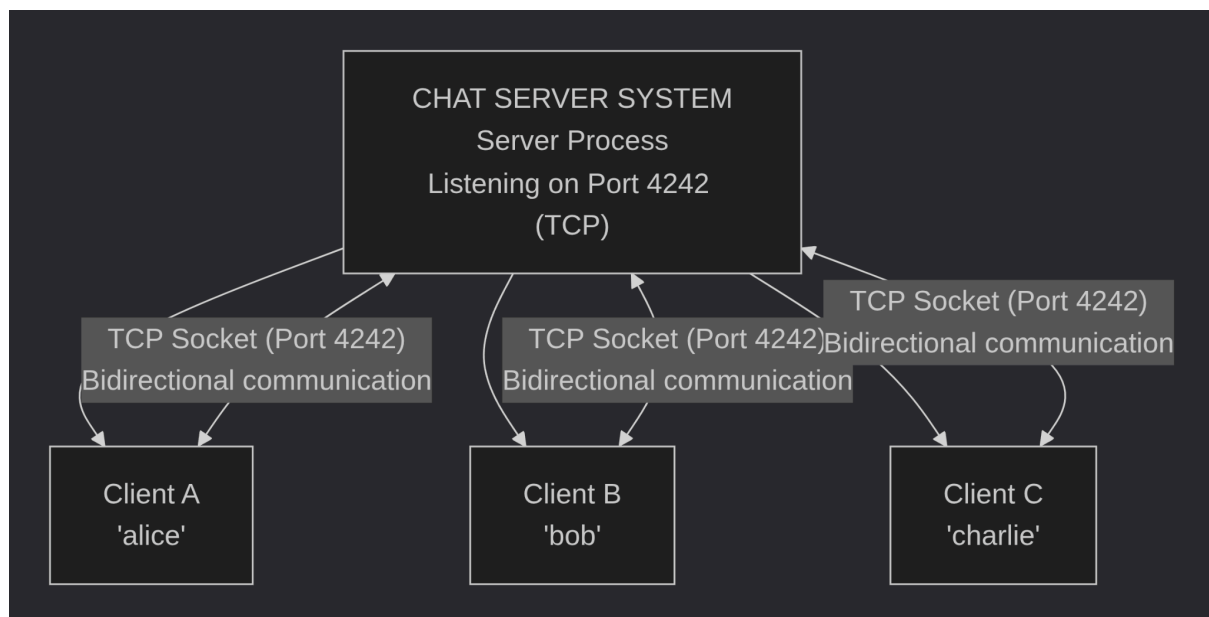
- `-h HOST` - Server hostname or IP address (default: localhost)
- `-p PORT` - Server port number (default: 4242)
- `-?` - Show help message

Client Commands (after starting):

- `/join <username>` - Join the chat with specified username (calls `send_join_message()` in `src/client.c`, joins the channel general by default)
- `/channel <name>` - Join an existing channel (sends `MSG_TYPE_CHANNEL_JOIN`)

- `/create <name>` - Create and join a new channel (sends `MSG_TYPE_CHANNEL_CREATE`)
- `/color <color>` - Change username color (red, orange, yellow, green, turquoise, blue, purple, white) (sends `MSG_TYPE_COLOR`)
- `/list` - List all available channels (sends `MSG_TYPE_CHANNEL_LIST`)
- `/quit` - Exit the client (calls `send_leave_message()`)
- `[any text]` - Send a chat message to current channel (calls `send_chat_message()`)

5.3 Architecture Diagram



Process Relationships:

- **Server Process:** Single process that listens on TCP port 4242, accepts client connections, and routes messages between clients in the same channels.
- **Client Processes:** Multiple independent processes that connect to the server, authenticate with usernames, and exchange messages through the server.
- **Relationship:** All clients connect to the single server. Clients do not connect directly to each other. The server acts as a message router.

Socket Connections:

- **Type:** TCP sockets (data order and correctness guaranteed unlike UDP)
- **Port:** 4242 (configurable via Makefile PORT variable)
- **Persistence:** Each client maintains one persistent TCP connection to the server for the duration of its session.
- **Concurrency:** Server handles multiple concurrent connections using `select()` system call.

Data Flow Through Connections: Each TCP socket carries message types bidirectionally between client and server, including authentication, chat messages, channel management, system notifications, and error messages.

5.4 Communication Protocol

All messages follow a binary format with an 8-byte header followed by a variable-length payload (defined in `include/protocol.h`):

```
typedef struct {  
  
    uint32_t type;    // Message type (MSG_TYPE_*)  
  
    uint32_t length;  // Payload length in bytes  
  
} message_header_t;
```

Message Types and Specifications:

1. JOIN (Client → Server)

- **Data:** `username` (64 bytes)
- **Semantics:** Join chat with username → added to "general"
- **Error:** Duplicate username → ERROR

2. CHAT (↔ Bidirectional)

- **Data:** `channel` (64) + `sender` (32) + `content` (1024) = 1120 bytes
- **Semantics:** Client → Server = send to channel; Server → Clients = broadcast (excluding sender)
- **Error:** Unauthenticated → ERROR

3. SERVER (Server → Client)

- **Data:** `message` (1024 bytes)
- **Semantics:** System notifications (welcome, join/leave announcements)
- **Error:** None (informational)

4. ERROR (Server → Client)

- **Data:** `error_msg` (1024 bytes)
- **Semantics:** Error notification
- **Error:** Client displays to user

5. ACK (Server → Client)

- **Data:** None (0 bytes)
- **Semantics:** Success confirmation

- **Error:** None

6. CHANNEL_JOIN (Client → Server)

- **Data:** `channel_name` (32 bytes)
- **Semantics:** Request to join channel
- **Error:** Invalid channel → ERROR

7. CHANNEL_CREATE (Client → Server)

- **Data:** `channel_name` (32 bytes)
- **Semantics:** Create new channel and join it
- **Error:** Duplicate name → ERROR

8. CHANNEL_LIST (Client → Server)

- **Data:** None
- **Semantics:** Request channel list
- **Error:** None (always succeeds)

9. COLOR (Client → Server)

- **Data:** `username` (64) + `color` (32) = 96 bytes
- **Semantics:** Change display color → server updates and notifies channel
- **Error:** Invalid color → ERROR

10. LEAVE (Client → Server)

- **Data:** None
- **Semantics:** Graceful disconnect → server removes client and notifies channel
- **Error:** None

5.5 Concurrency Model

Server Concurrency:

The server uses a single-threaded event loop with `select()` to handle multiple clients concurrently:

1. File Descriptor Management:

- `server_fd`: Listening socket for new connections
- `client_fds[]`: One socket per connected client
- `master_set`: fd_set containing all monitored file descriptors

2. Event Loop (`server_run()` in `src/server.c:915-940`):

```
while (1) {
```

```

read_fds = state->master_set; // Copy fd_set

select(state->max_fd + 1, &read_fds, NULL, NULL, NULL);

for (fd = 0; fd <= state->max_fd; fd++) {

    if (FD_ISSET(fd, &read_fds)) {

        if (fd == state->server_fd) {

            handle_new_connection(state); // New client

        } else {

            handle_client_message(state, fd); // Existing client

        }

    }

}

}

```

English description:

1. Copy the master file descriptor set
2. Wait for activity on any socket using `select()`
3. Check each socket for activity
4. If listening socket: accept new client connection
5. If client socket: read and process client message
6. Repeat

Key point: Single thread handles all clients concurrently via event-driven I/O.

3. Non-blocking Design:

- Server never blocks on any single client operation
- All socket operations are non-blocking via `select()`
- Message processing is atomic inside the event loop

Client Concurrency:

Each client uses `select()` to handle both user input and server messages concurrently:

1. File Descriptor Monitoring:

- `STDIN_FILENO` (0): User input from terminal
- `sockfd`: Server socket for incoming messages

2. Race Condition Prevention:

- User input processed before server messages in each loop
- Line clearing (`\r\x1b[2K`) prevents duplicate message display
- Atomic message display ensures clean output

5.6 Error Handling and Robustness

1. Client Disconnection Mid-Message

Scenario: Client disconnects unexpectedly while server is reading a message.

Handling: `receive_message_header()` or `receive_message_data()` (in `src/protocol.c`) returns -1 or 0. Server detects EOF/error in `handle_client_message()` (line ~ in `src/server.c`), calls `client_remove()` to clean up resources, and broadcasts leave notification to channel.

Code Reference: `handle_client_message()` in `src/server.c` checks return values from receive functions and calls `client_remove()` on error.

2. Malformed Command from Client

Scenario: Client sends invalid command or malformed message.

Handling: Server validates message structure in `handle_client_message()` (in `src/server.c`) and command syntax. Invalid requests receive `MSG_TYPE_ERROR` response via `send_error_message()`. Client remains connected.

Code Reference: Command parsing in `process_user_input()` (in `src/client.c`) and message validation in `handle_client_message()` (in `src/server.c`).

3. Network Failure During Broadcast

Scenario: Server tries to broadcast to a client that has disconnected.

Handling: `send_message()` (in `src/protocol.c`) returns -1 on write error. Server detects failed send in `channel_broadcast()` (in `src/server.c`), calls `client_remove()` for that client, and continues broadcasting to remaining clients.

Code Reference: `channel_broadcast()` in `src/server.c` checks `send_message()` return value and handles errors.

5.7 Team Contributions

Only me. AI was partially used for this report, and in some parts of the code as specified in comments.